

Компьютерный практикум по статистическому анализу данных

**Лабораторная работа №1. Julia: установка, настройка и
основные принципы**

Ведьмина Александра Сергеевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Язык Julia и интерактивные вычисления	7
3.2	Типы и преобразование данных	7
3.3	Векторы и матрицы	8
3.4	Воспроизводимый проект	8
4	Выполнение лабораторной работы	9
4.1	Подготовка рабочего окружения	9
4.2	Создание проекта DrWatson	10
4.3	Проверка пакетов	11
4.4	Реализация базовых примеров Julia	12
4.5	Ввод, вывод и преобразование строк	14
4.6	Математические операции и массивы	15
4.7	Параметрический расчёт	16
4.8	Генерация производных форматов	17
4.9	Автоматические тесты	20
5	Результаты	21
5.1	Численные результаты	21
5.2	Результаты решения ОДУ	22
5.3	Полученные артефакты	23
6	Обсуждение результатов	24
7	Заключение	25
	Список литературы	26

Список иллюстраций

4.1	Проверка расположения и версии Julia	9
4.2	Установка Jupyter Lab в виртуальное окружение	10
4.3	Проверка зарегистрированных ядер Jupyter	10
4.4	Создание структуры проекта средствами DrWatson	11
4.5	Повторная безопасная проверка структуры проекта	11
4.6	Проверка активного проекта, каталогов и зависимостей	12
4.7	Результаты примеров с типами Julia	14
4.8	Параметрический расчёт, сохранение данных и решение ОДУ	17
4.9	Генерация Julia-кода, notebook и Quarto-документа	18
4.10	Выполнение всех ячеек при сборке Quarto-документа	18
4.11	Начальная часть выполненного Jupyter Notebook	19
4.12	Заключительная часть выполненного Jupyter Notebook	19
4.13	Успешное выполнение автоматических тестов	20
5.1	Зависимость нормы масштабированной матрицы от параметра	22

Список таблиц

5.1	Результаты вычисления αAv	21
5.2	Численное решение уравнения $u' = -u$	22

1 Цель работы

Подготовить рабочее пространство и инструментарий для работы с языком программирования Julia, освоить работу с Julia в Jupyter Lab и на простейших примерах познакомиться с базовым синтаксисом языка.

2 Задание

В ходе лабораторной работы требовалось:

1. установить Julia и Jupyter Lab, подключить ядро IJulia;
2. создать воспроизводимый вычислительный проект в структуре DrWatson;
3. повторить примеры определения типов, преобразования значений, функций, векторов и матриц;
4. изучить функции ввода-вывода `read`, `readline`, `readlines`, `readdlm`, `print`, `println`, `show` и `write`;
5. изучить функцию `parse` и безопасное преобразование `tryparse`;
6. выполнить примеры математических, сравнительных и логических операций;
7. выполнить операции над векторами и матрицами;
8. провести расчёт для нескольких значений параметра, сохранить результаты и построить график;
9. получить из единого literate-источника исполняемый Julia-файл, notebook и Quarto-документ;
10. проверить проект автоматическими тестами.

3 Теоретическое введение

3.1 Язык Julia и интерактивные вычисления

Julia является высокоуровневым языком программирования, ориентированным на численные и научные вычисления. Язык поддерживает динамическую диспетчеризацию, параметрические типы, эффективные массивы и непосредственную работу с математическими выражениями [6].

Jupyter Lab организует работу в виде notebook-документа, объединяющего текст, программный код, результаты выполнения и визуализации. Пакет IJulia регистрирует Julia как доступное ядро Jupyter [3; 5].

3.2 Типы и преобразование данных

Функция `typeof` возвращает тип значения. Для целочисленных типов функции `typemin` и `typemax` задают границы представимого диапазона. Значения `Inf`, `-Inf` и `NaN` имеют вещественный тип и используются для представления бесконечности и неопределённого результата.

Преобразование данных выполняется конструктором требуемого типа или функцией `convert`. Функция `promote` приводит несколько значений к общему типу, если такое преобразование возможно. Функция `parse` преобразует строку в заданный тип, а `tryparse` вместо исключения возвращает `nothing` при некорректном вводе.

3.3 Векторы и матрицы

Одномерный массив Julia задаёт вектор, а двумерный — матрицу. Индексация начинается с единицы. Оператор `*` реализует матричное умножение, `.*` — поэлементное, апостроф выполняет сопряжённое транспонирование, а функция `dot` вычисляет скалярное произведение.

В параметрической части работы для матрицы A , вектора v и параметра α вычислялись

$$A_\alpha = \alpha A, \quad r_\alpha = A_\alpha v, \quad n_\alpha = \|A_\alpha\|_F.$$

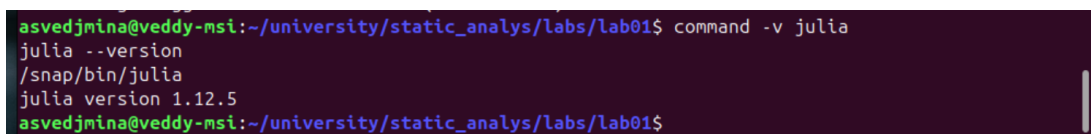
3.4 Воспроизводимый проект

Пакет `DrWatson` предоставляет стандартные каталоги для кода, данных и графиков и позволяет получать пути независимо от текущего рабочего каталога [1]. Пакет `Literate` создаёт из одного комментированного Julia- файла чистый сценарий, Jupyter Notebook и документ Quarto [2; 4]. Фиксация зависимостей в `Project.toml` и `Manifest.toml` обеспечивает повторяемость программного окружения.

4 Выполнение лабораторной работы

4.1 Подготовка рабочего окружения

Сначала была проверена установленная версия Julia. В работе использовалась Julia 1.12.5 (Рисунок 4.1).



```
asvedjmina@veddy-msi:~/university/static_analys/labs/lab01$ command -v julia
julia --version
/snap/bin/julia
julia version 1.12.5
asvedjmina@veddy-msi:~/university/static_analys/labs/lab01$
```

Рисунок 4.1: Проверка расположения и версии Julia

Jupyter Lab был установлен в отдельное Python-окружение, после чего это окружение было активировано в терминале (Рисунок 4.2).

```

asvedjmina@veddy-msi:~/university/static_analys/labs/lab01$ python3 -m venv ~/.venvs/jupyter
source ~/.venvs/jupyter/bin/activate
python -m pip install --upgrade pip
python -m pip install jupyterlab
jupyter lab --version
Requirement already satisfied: pip in /home/asvedjmina/.venvs/jupyter/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 414.5 kB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
  Successfully installed pip-26.2.1
Collecting jupyterlab
  Downloading jupyterlab-4.6.3-py3-none-any.whl.metadata (16 kB)
Collecting async_lru>=1.0.0 (from jupyterlab)
  Downloading async_lru-2.3.0-py3-none-any.whl.metadata (7.6 kB)
Collecting httpx<1,>=0.25.0 (from jupyterlab)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting ipykernel!=6.30.0,>=6.5.0 (from jupyterlab)
  Downloading ipykernel-7.3.0-py3-none-any.whl.metadata (4.5 kB)
Collecting jinja2>=3.0.3 (from jupyterlab)

```

Рисунок 4.2: Установка Jupyter Lab в виртуальное окружение

Пакет IJulia был связан с установленным Jupyter. Команда `jupyter kernelspec list` подтвердила наличие ядра Julia (Рисунок 4.3).

```

[30] _start()
@ Base ./client.jl:550
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01$ jupyter kernelspec list
Available kernels:
  python3          /home/asvedjmina/.venvs/jupyter/share/jupyter/kernels/python3
  julia-1.12       /home/asvedjmina/.local/share/jupyter/kernels/julia-1.12
  julia-lab04-1.12 /home/asvedjmina/.local/share/jupyter/kernels/julia-lab04-1.12
  julia-project-1.12 /home/asvedjmina/.local/share/jupyter/kernels/julia-project-1.12
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01$

```

Рисунок 4.3: Проверка зарегистрированных ядер Jupyter

4.2 Создание проекта DrWatson

Проект был создан программно функцией `initialize_project`. В результате появились `Project.toml`, `Manifest.toml` и стандартные каталоги `data`, `src`, `scripts`, `plots`, `notebooks` и `test` (Рисунок 4.4).

```
julia> initialize_project(
    project_name;
    authors=["BAWE ФИО"],
    git=false,
    placeholder=true,
)

Activating new project at `~/university/static_analys/labs/lab01/project`
Updating registry at `~/julia/registries/General.toml`
Resolving package versions...
Updating `~/university/static_analys/labs/lab01/project/Project.toml`
[634d3b9d] + DrWatson v2.19.1
Updating `~/university/static_analys/labs/lab01/project/Manifest.toml`
[0b6fb165] + ChunkCodecCore v1.0.2
[4c0bbe4] + ChunkCodecLibZlib v1.1.0
[55437552] + ChunkCodecLibZstd v1.0.0
[634d3b9d] + DrWatson v2.19.1
[5789e2e9] + FileIO v1.20.0
[076d061b] + HashArrayMappedTries v0.2.0
[033835bb] + JLD2 v0.6.6
[692b3bcd] + JLLWrappers v1.8.0
[1914dd2f] + MacroTools v0.5.16
[bae558e1] + OrderedCollections v2.0.1
```

Рисунок 4.4: Создание структуры проекта средствами DrWatson

Сценарий `setup_project.jl` сделан идемпотентным: повторный запуск обнаруживает существующий проект и не перезаписывает пользовательские файлы (Рисунок 4.5).

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01/project$ cd /home/asvedjmina
/home/asvedjmina/university/static_analys/labs/lab01
julia setup_project.jl
cd project
Проект DrWatson уже существует:
/home/asvedjmina/university/static_analys/labs/lab01/project
Существующие файлы не изменены.

Следующий шаг:
cd /home/asvedjmina/university/static_analys/labs/lab01/project
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01/project$
```

Рисунок 4.5: Повторная безопасная проверка структуры проекта

Зависимости проекта были установлены с помощью `Pkg`. Для медленного соединения в загрузчике был увеличен допустимый интервал отсутствия передачи данных, а `IJulia` настроена на системный `Jupyter`.

4.3 Проверка пакетов

Сценарий `scripts/test_setup.jl` проверяет пути проекта и загрузку всех требуемых пакетов: `DrWatson`, `DifferentialEquations`, `Plots`, `DataFrames`, `CSV`, `JLD2`,

Literate, IJulia, BenchmarkTools и Quarto (Рисунок 4.6).

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01/project$ julia --project=. scripts/test_setup.jl
Проверка рабочего окружения лабораторной работы № 1
=====
Версия Julia:      1.12.5
Активный проект:  /home/asvedjmina/university/static_analys/labs/lab01/project/Project.toml
Корень проекта:   /home/asvedjmina/university/static_analys/labs/lab01/project
Каталог данных:  /home/asvedjmina/university/static_analys/labs/lab01/project/data
Каталог скриптов: /home/asvedjmina/university/static_analys/labs/lab01/project/scripts
Каталог исходников: /home/asvedjmina/university/static_analys/labs/lab01/project/src
Каталог графиков: /home/asvedjmina/university/static_analys/labs/lab01/project/plots
```

Рисунок 4.6: Проверка активного проекта, каталогов и зависимостей

4.4 Реализация базовых примеров Julia

Основной файл `scripts/lab01_literate.jl` является одновременно исполняемым сценарием и источником документации. В нём определяются типы чисел, специальные вещественные значения и границы целочисленных диапазонов:

```
values = (3, 3.5, 3 / 3.5, 3 + 4im, pi)
for value in values
    println(repr(value), " имеет тип ", typeof(value))
end

for T in [Int8, Int16, Int32, Int64, Int128,
          UInt8, UInt16, UInt32, UInt64, UInt128]
    println(
        lpad(string(T), 7),
        ": [", typemin(T), ", ",
        typemax(T), "]",
    )
end
```

Преобразование типов и два способа определения функций реализованы

следующим образом:

```
direct_conversion = (  
    Int64(2.0), Char(65),  
)  
generic_conversion = (  
    convert(Int64, 2.0),  
    convert(Char, 65),  
)  
promoted = promote(Int8(1), Float16(4.5), Float32(4.1))  
  
function square(x)  
    return x^2  
end  
cube(x) = x^3
```

Фрагмент вывода сценария с типами, диапазонами и преобразованиями представлен ниже (Рисунок 4.7).

```

(jupyter) asvedjmina@veddy-nsi:~/university/static_analys/labs/lab01/project$ julia --project=. scripts/lab01_literate.jl
Версия Julia: 1.12.5
Дата выполнения: 2026-09-08
Активный проект: /home/asvedjmina/university/static_analys/labs/lab01/project/Project.toml
Корень проекта: /home/asvedjmina/university/static_analys/labs/lab01/project
3 имеет тип Int64
3.5 имеет тип Float64
0.8571428571428571 имеет тип Float64
3 + 4im имеет тип Complex{Int64}
π имеет тип Irrational{:π}
Inf имеет тип Float64
-Inf имеет тип Float64
NaN имеет тип Float64
Inf имеет тип Float64
NaN имеет тип Float64
Int8: [-128, 127]
Int16: [-32768, 32767]
Int32: [-2147483648, 2147483647]
Int64: [-9223372036854775808, 9223372036854775807]
Int128: [-170141183460469231731687303715884105728, 170141183460469231731687303715884105727]
UInt8: [0, 255]
UInt16: [0, 65535]
UInt32: [0, 4294967295]
UInt64: [0, 18446744073709551615]
UInt128: [0, 340282366920938463463374607431768211455]
Прямое преобразование: (2, 'A')
Преобразование convert: (2, 'A')
Преобразование Bool: (true, false)
Результат promote: (1.0f0, 4.5f0, 4.1f0)
Типы после promote: (Float32, Float32, Float32)
square(4) = 16
cube(3) = 27
Вектор-строка: [4 7 6]

```

Рисунок 4.7: Результаты примеров с типами Julia

4.5 Ввод, вывод и преобразование строк

Для демонстрации файлового ввода-вывода были созданы текстовый, бинарный и табличный файлы. Рассмотрены чтение всего файла, одной строки и массива строк, а также запись байтов и таблицы:

```

whole_text = read(text_file, String)

open(text_file, "r") do io
    println("Результат readline: ", readline(io))
end

all_lines = readlines(text_file)

bytes = UInt8[
    0x41, 0x42, 0x43, 0x0a,
]

```

```

bytes_written = write(binary_file, bytes)
writedlm(table_file, numeric_table, ',')
table_from_file = readdlm(table_file, ',', Float64)

```

Функции `print`, `println` и `show` были сопоставлены по формату вывода. Преобразование строк проверено на целых, вещественных и логических значениях, а также на числах с основаниями 2 и 16:

```

parse{Int, "42"}
parse{Float64, "3.1415"}
parse{Bool, "true"}
parse{Int, "1010"; base=2}
parse{Int, "ff"; base=16}
tryparse{Int, "abc"}

```

4.6 Математические операции и массивы

Проверены сложение, вычитание, умножение, обычное и целочисленное деление, остаток, степень, квадратный корень, сравнения и логические операции. Отдельно выполнены операции с векторами и матрицами:

```

v + w
v - w
dot(v, w)
transpose(v)
2 .* v
A + B
A - B
A * v

```

```
A * B
A .* B
```

4.7 Параметрический расчёт

Вычислительная функция вынесена в `src/lab01.jl` и возвращает параметр, норму масштабированной матрицы и полученный вектор:

```
function calculate_for_parameter(alpha, matrix, vector)
    scaled_matrix = alpha .* matrix
    result_vector = scaled_matrix * vector
    return (
        alpha=alpha,
        matrix_norm=norm(scaled_matrix),
        result=result_vector,
    )
end
```

Расчёт проведён для $\alpha \in \{0.5, 1.0, 2.0\}$. Результаты сохранены в двух файлах:

- `parameter_results.csv` в каталоге `data/exp_pro`;
- `parameter_results.jld2` в каталоге `data/sims`.

Итоговый вывод сценария представлен ниже (Рисунок 4.8).

```

7.0 7.0 6.0
A * v = [14.0, 32.0, 53.0]
A * B =
3x3 Matrix{Float64}:
 4.0  9.0 17.0
13.0 21.0 38.0
22.0 34.0 63.0
Поэлементное A .* B =
3x3 Matrix{Float64}:
 2.0  0.0  3.0
 4.0 15.0 12.0
 0.0  8.0 40.0
alpha=0.5, норма=8.718, результат=[7.0, 16.0, 26.5]
alpha=1.0, норма=17.436, результат=[14.0, 32.0, 53.0]
alpha=2.0, норма=34.871, результат=[28.0, 64.0, 106.0]
Таблица результатов для набора параметров:
3x5 DataFrame
  Row  alpha  matrix_norm  result_1  result_2  result_3
    Float64  Float64      Float64  Float64  Float64
1      0.5      8.7178      7.0      16.0      26.5
2      1.0     17.4356     14.0     32.0     53.0
3      2.0     34.8712     28.0     64.0    106.0
CSV сохранён: /home/asvedjmina/university/static_analys/labs/lab01/project/data/exp_pro/parameter_results.csv
JLD2 сохранён: /home/asvedjmina/university/static_analys/labs/lab01/project/data/sims/parameter_results.jld2
График сохранён: /home/asvedjmina/university/static_analys/labs/lab01/project/plots/parameter_norms.png
Решение ОДУ в узлах: [1.0, 0.3678796381978341, 0.13533581256777455, 0.04978770787351412, 0.018315275151814518, 0.006738503575123539]
Оценка времени sum(v), секунд: 2.349e-9
(jupyter) asvedjmina@veddy-nsi:~/university/static_analys/labs/lab01/project$ julia --project=. scripts/generate_outputs.jl

```

Рисунок 4.8: Параметрический расчёт, сохранение данных и решение ОДУ

Дополнительно решена задача Коши

$$\frac{du}{dt} = -u, \quad u(0) = 1,$$

с помощью DifferentialEquations.jl. Полученные значения совпадают с аналитическим решением $u(t) = e^{-t}$ в пределах численной точности.

4.8 Генерация производных форматов

Сценарий scripts/generate_outputs.jl сформировал три представления работы:

- generated/lab01_code.jl — чистый исполняемый код;
- generated/lab01_notebook.ipynb — исполненный Jupyter Notebook;
- generated/lab01_quarto.qmd — документ Quarto.

Запуск генератора представлен ниже (Рисунок 4.9).

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01/project$ julia --project=. scripts/generate_outputs.jl
Исходный файл: /home/asvedjmina/university/static_analys/labs/lab01/project/scripts/lab01_literate.jl
Каталог результатов: /home/asvedjmina/university/static_analys/labs/lab01/project/generated
[ Info: generating plain script file from `~/university/static_analys/labs/lab01/project/scripts/lab01_literate.jl`
[ Info: writing result to `~/university/static_analys/labs/lab01/project/generated/lab01_code.jl`
[ Info: generating notebook from `~/university/static_analys/labs/lab01/project/scripts/lab01_literate.jl`
[ Info: executing notebook `lab01_notebook.ipynb`
[ Info: writing result to `~/university/static_analys/labs/lab01/project/generated/lab01_notebook.ipynb`
[ Info: generating markdown page from `~/university/static_analys/labs/lab01/project/scripts/lab01_literate.jl`
[ Info: writing result to `~/university/static_analys/labs/lab01/project/generated/lab01_quarto.qmd`

Сгенерирован чистый код: /home/asvedjmina/university/static_analys/labs/lab01/project/generated/lab01_code.jl
Сгенерирован notebook: /home/asvedjmina/university/static_analys/labs/lab01/project/generated/lab01_notebook.ipynb
Сгенерирован Quarto-документ: /home/asvedjmina/university/static_analys/labs/lab01/project/generated/lab01_quarto.qmd
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01/project$
```

Рисунок 4.9: Генерация Julia-кода, notebook и Quarto-документа

Quarto последовательно выполнил все 18 кодовых ячеек и сформировал самодостаточный HTML-документ (Рисунок 4.10).

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01/project$ quarto render generated/lab01_quarto.qmd --to html

Starting julia-1.12 kernel...Done

Executing 'lab01_quarto.ipynb'
Cell 1/18: '...Done
Cell 2/18: '...Done
Cell 3/18: '...Done
Cell 4/18: '...Done
Cell 5/18: '...Done
Cell 6/18: '...Done
Cell 7/18: '...Done
Cell 8/18: '...Done
Cell 9/18: '...Done
Cell 10/18: '...Done
Cell 11/18: '...Done
Cell 12/18: '...Done
Cell 13/18: '...Done
Cell 14/18: '...Done
Cell 15/18: '...Done
Cell 16/18: '...Done
Cell 17/18: '...Done
Cell 18/18: '...Done

pandoc
to: html
output-file: lab01_quarto.html
standalone: true
section-divs: true
html-math-method: mathjax
wrap: none
default-image-extension: png
```

Рисунок 4.10: Выполнение всех ячеек при сборке Quarto-документа

Notebook был открыт в Jupyter Lab. Начальная и заключительная части выполненного документа представлены отдельно: Рисунок 4.11 и Рисунок 4.12.

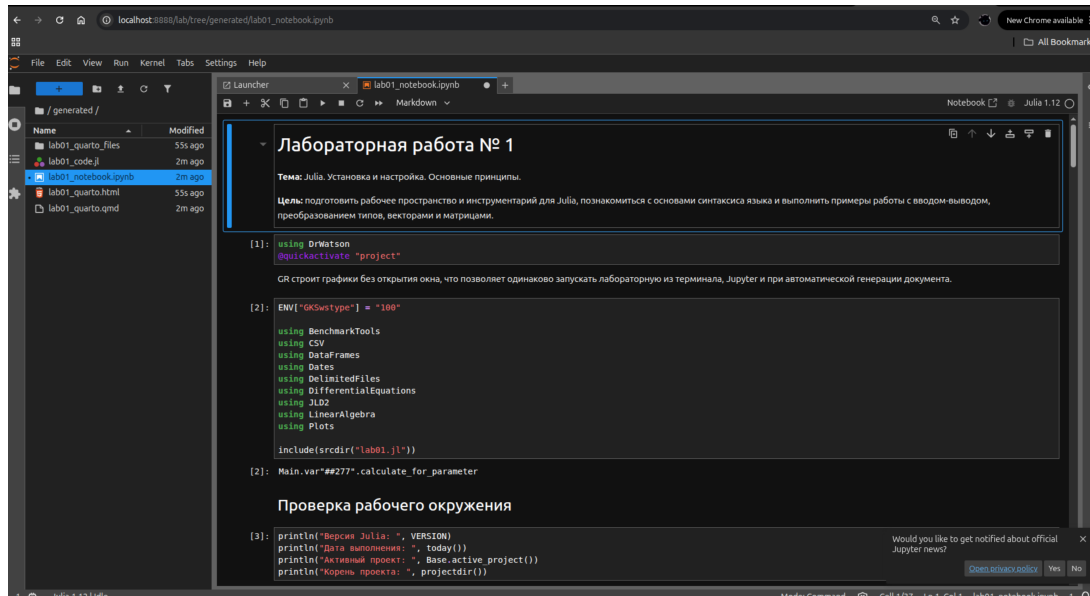


Рисунок 4.11: Начальная часть выполненного Jupyter Notebook

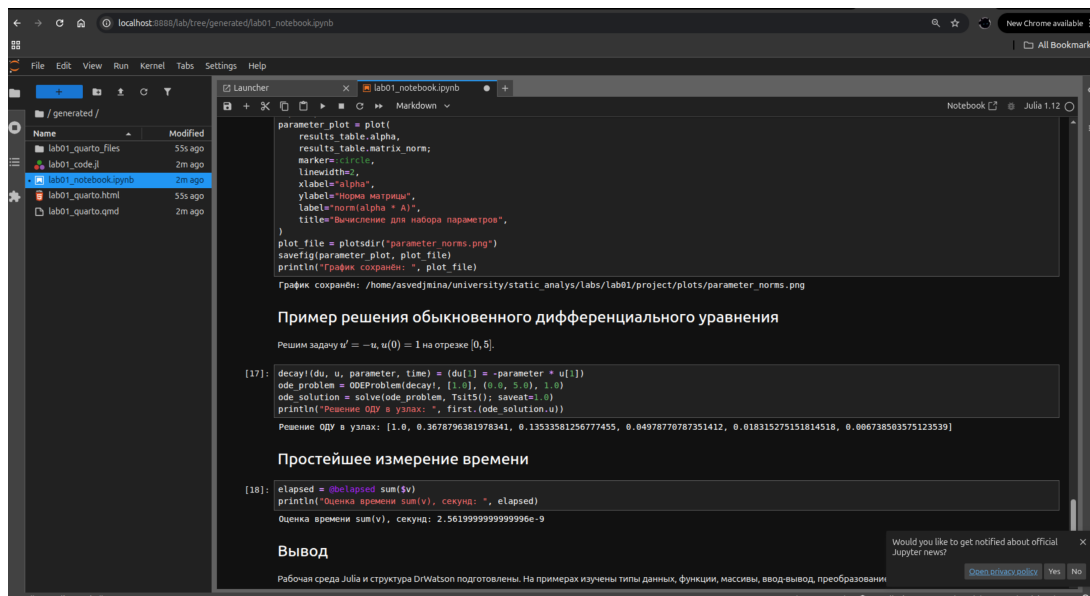


Рисунок 4.12: Заключительная часть выполненного Jupyter Notebook

4.9 Автоматические тесты

Тесты проверяют три численных результата функции `calculate_for_parameter` и четыре элемента структуры проекта. Все семь проверок завершились успешно (Рисунок 4.13).

```
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01/project$ julia --project=. test/runtests.jl
Starting tests
Test Summary:      | Pass Total Time
Lab 01 calculations |    3     3 0.5s
Test Summary:      | Pass Total Time
Project structure |    4     4 0.0s

Test took total time of:
0.014 minutes
(jupyter) asvedjmina@veddy-msi:~/university/static_analys/labs/lab01/project$
```

Рисунок 4.13: Успешное выполнение автоматических тестов

5 Результаты

5.1 Численные результаты

Сводные результаты параметрического расчёта представлены ниже (Таблица 5.1).

Таблица 5.1: Результаты вычисления αAv

α	$\ \alpha A\ _F$	Первый элемент	Второй элемент	Третий элемент
0,5	8,7178	7,0	16,0	26,5
1,0	17,4356	14,0	32,0	53,0
2,0	34,8712	28,0	64,0	106,0

Линейная зависимость нормы матрицы от α представлена ниже (Рисунок 5.1). При удвоении параметра с 1,0 до 2,0 норма и каждый элемент результирующего вектора также удваиваются.

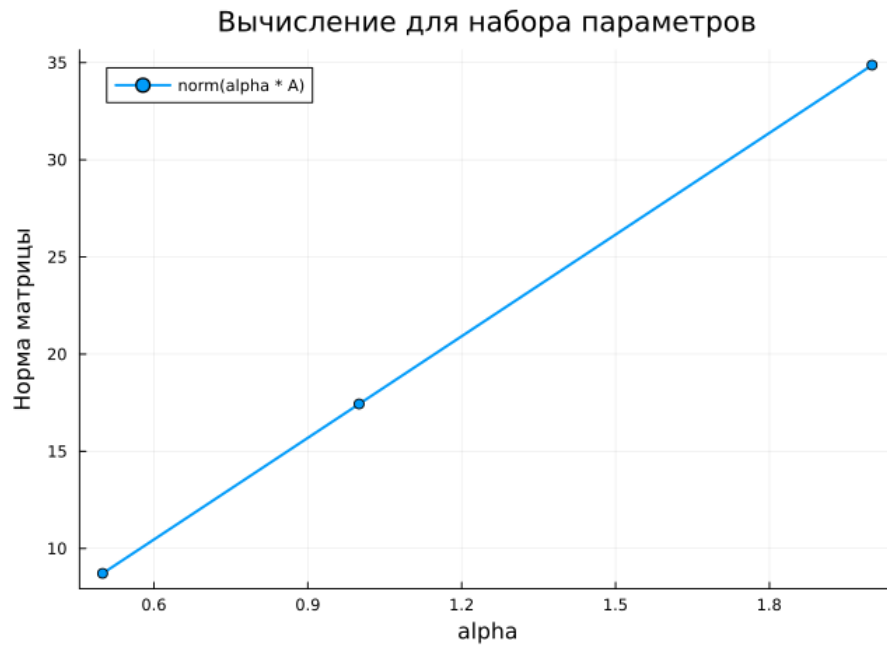


Рисунок 5.1: Зависимость нормы масштабированной матрицы от параметра

5.2 Результаты решения ОДУ

Для моментов времени от 0 до 5 с шагом 1 получены значения:

Таблица 5.2: Численное решение уравнения $u' = -u$

t	Численное значение $u(t)$
0	1,000000
1	0,367880
2	0,135336
3	0,049788
4	0,018315
5	0,006739

5.3 Полученные артефакты

По результатам работы сформированы:

- воспроизводимое окружение `Project.toml` и `Manifest.toml`;
- `literate`-источник и отдельный вычислительный модуль;
- выполненный `notebook` без ошибочных ячеек;
- чистый `Julia`-сценарий и `Quarto`-документ;
- `CSV`-таблица, `JLD2`-файл и `PNG`-график;
- автоматические тесты вычислений и структуры проекта.

6 Обсуждение результатов

Все задания лабораторной работы воспроизведены в едином сценарии. Использование DrWatson исключает жёсткое задание относительных путей, а Manifest.toml фиксирует конкретные версии зависимостей. Literate-подход устраняет расхождение между кодом в notebook и кодом, запускаемым из терминала: оба представления получены из одного источника.

Параметрический эксперимент подтверждает свойства умножения матрицы на скаляр: при положительном α норма Фробениуса и результирующий вектор изменяются линейно. Решение тестового ОДУ соответствует экспоненциальному убыванию. Повторный запуск сценария перезаписывает только производные результаты и потому не требует ручной очистки проекта.

7 Заключение

В ходе лабораторной работы:

1. установлены и настроены Julia, Jupyter Lab и IJulia;
2. создан воспроизводимый проект DrWatson и установлены необходимые пакеты;
3. изучены базовые типы, преобразования, функции, ввод-вывод и операции с массивами;
4. реализован параметрический расчёт с сохранением данных и построением графика;
5. решён пример обыкновенного дифференциального уравнения;
6. из одного literate-источника получены Julia-код, notebook и Quarto-документ;
7. все 18 ячеек notebook выполнены без ошибок, а все 7 автоматических тестов пройдены.

Таким образом, цель работы достигнута: рабочее окружение подготовлено, основные конструкции Julia изучены на выполненных примерах, а результаты оформлены в воспроизводимом вычислительном проекте.

Список литературы

1. DrWatson: the perfect sidekick for your scientific inquiries / G. Datseris [et al.] // Journal of Open Source Software. — 2020. — Vol. 5, no. 54. — P. 2673. — DOI: 10.21105/joss.02673.
2. *Ekre F.* Literate.jl Documentation. — URL: <https://fredrikekre.github.io/Literate.jl/> (visited on 09/08/2026).
3. *IJulia contributors.* IJulia Documentation. — URL: <https://julialang.github.io/IJulia.jl/stable/> (visited on 09/08/2026).
4. *Posit Software, PBC.* Quarto Documentation. — URL: <https://quarto.org/docs/> (visited on 09/08/2026).
5. *Project Jupyter.* JupyterLab Documentation. — URL: <https://jupyterlab.readthedocs.io/> (visited on 09/08/2026).
6. *The Julia Project.* The Julia Language Documentation. — URL: <https://docs.julialang.org/> (visited on 09/08/2026).